Computational Schlieren Imaging Using Geometric Ray Tracing for Blast Waves

1.0 Introduction

Geometric ray tracing is used to create schlieren images from CFD simulations and to compare with experimental data. For this project, geometric ray tracing was used to create schlieren images of blast waves.

Geometric ray tracing simulates the behaviour of individual photons as they travel through mediums of varying indices of refraction. Each ray represents an individual photon and geometric ray tracing traces the path of each photon as it moves through the medium. Geometric ray tracing requires the density distribution of a particular fluid that the ray travels through. Using the density of the flow field, an index of refraction (IOR) distribution can be calculated. The path the light rays take depends on the IOR of the flow field and is traced through the field using geometric ray tracing. The ray is traced through the field until it hits a simulated screen. Each pixel of the produced schlieren image can be represented by a single ray. The intensity of each pixel is related to the displacement of the final location of the ray on the screen from its original starting location. The intensity distribution of the schlieren image is dependent on the particular knife edge that is simulated.

2.0 Data Collection and Processing

2.1 Collecting and Processing Data for Spherical Blast Waves

The density distributions of the blast waves were needed in order to find the IOR distribution of the blast waves. The density distributions for the spherical blasts were given by the paper Laser Energy Deposition in Quiescent Air by H. Yan [1]. The density distributions were given at 10 us intervals starting from 10 us to 40 us.

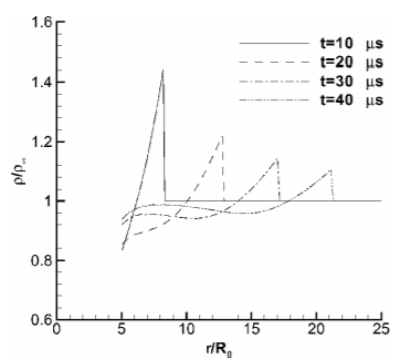


Fig. 17 Density vs radius for $E = 112$ mJ.

Figure 1: Spherical Density Distributions

Using the values of R_o and p_∞ found in the paper, 0.9 cm and 1.2 kg/m^3, density in terms of radius can be found. The four curves were extracted from the paper using WebPlotDigitizer[2]. The dimensionless radius was multiplied by R_o in order to get the radius and the dimensionless density was multiplied by p_∞ in order to get the corresponding density. In order to convert from density to IOR, the Gladstone-Dale relation is used[3].

$$n - 1 = Kp.$$

Equation 1 Gladstone-Dale Relation

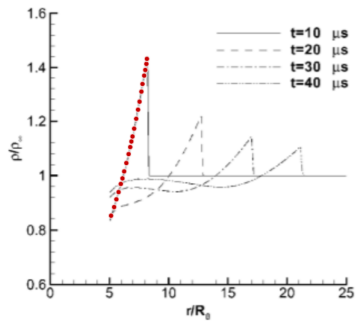


Figure 2: WebPlotDigitizer Data Extraction

n represents the IOR, p represents density of the gas and K is the Gladstone-Dale constant which is approximately 0.226 cm^3/g for air[3]. After some converting the values to the same units, the equations are manipulated to find the radius and corresponding IOR.

2.2 Collecting and Processing Data for Cylindrical Blast Waves

The density distributions for the cylindrical blast waves were given by the paper Shock Waves from Line Sources by M. Plooster[4]. In the paper, there are many density distributions found at different times shown in different plots[A1]. The times were given as dimensionless times τ which can be represented as shown below.

$$\tau = t\, a_o / R_o$$

Equation 2

One density distribution (the last one) was extracted from each plot for a total of 8 distributions using WebPlotDigitizer[2]. The reason was to have the density distributions at a variety of times so changes in the schlieren image would be visible.

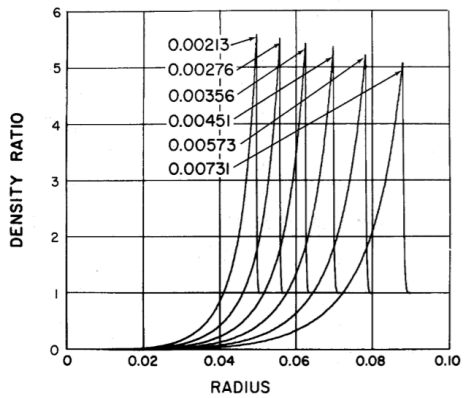


Fig. 5 (a and b) Density ratio (ρ/ρ_o) vs radius (η) for line source energy release in ideal gas, at indicated times after energy release.

Figure 3: Sample Density Plot from Paper

The units in the density plot are the dimensionless radius and the dimensionless density. In the paper it states that the characteristic radius R_o can be calculated as in the equation X. The characteristic radius R_o is used as in equation Y to find the dimensionless radius. By multiplying the dimensionless radius in the graph by the characteristic radius the actual radius can be found.

$$R_o = (4E_o/b\gamma p_o)^{\frac{1}{2}}$$

Equation 3

$$\eta = r/R_o$$

Equation 4

The density distribution was found by multiplying the density ratio in the graph by the ambient density of air which is 1.2754 kg/m^3. The constants b and γ were given by the paper to be 3.94 and 1.4 respectively. p_o is the ambient pressure which was set to 101325 Pa for air at sea level. E_o which is the energy per unit length of explosive was set to be a constant value of 1 J/m which is just a reference value that can be changed depending on the energy of the blast desired.

The data from WebPlotDigitizer was first graphed in Plotly[5] using one of its commands. The data points were then copied over into MATLAB. The data was saved in separate .mat files for different blast waves [A2][A3]. The data could then be loaded directly into the program. The data would later be used for the data selection function and would be useful for plotting.

3.0 Ray Tracing

3.1 Ray Tracing Description

The ray tracing code was written and tested using MATLAB. In order to test the functionality of the ray tracing code, the output of the ray tracing code was compared to a ray tracing benchmark. The benchmark is from the research paper “Simulating schlieren and shadowgraph images from LES data” [6]. The benchmark was derived from geometric ray tracing performed on a simple piecewise linear IOR field.

$$n(y,z) = \begin{cases} n_0 + \varepsilon\left(1 - \frac{2|y|}{b}\right) & y \le \left|\frac{b}{2}\right|; \quad z < \Delta z \\ n_0 & y > \left|\frac{b}{2}\right|; \quad z > \Delta z \end{cases}$$

Figure 4: Piecewise Linear IOR Field Used to Create a Geometric Ray Tracing Benchmark

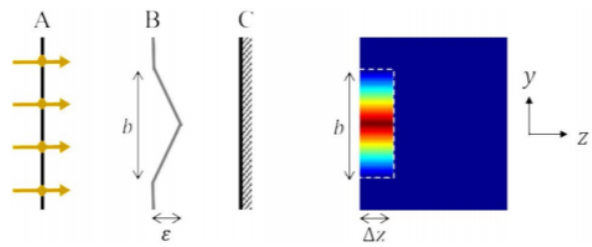


Fig. 7 Left: outline of the test setup, (A) an incident plane wave/ straight rays, (B) the variation in the IoR field, and (C) the screen on which the image will form. Right: contours of index of refraction in the test setup, dark blue represents the undisturbed state. The white dashed line shows the edge of the perturbed region

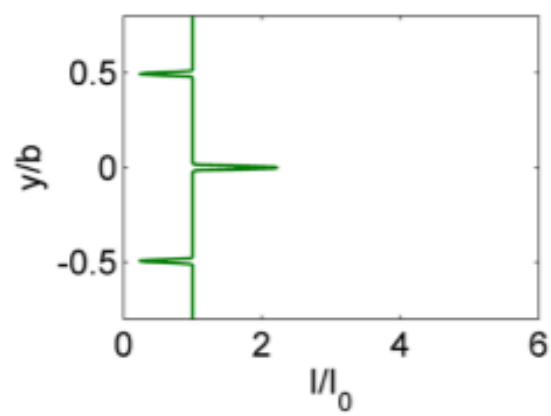


Figure 5: Description of the Benchmark Setup

Figure 6: Ray Tracing Benchmark Results

The ray tracing benchmark results were converted into data points [A4] using an online application called WebPlotDigitizer[2]. The ray tracing benchmark results were plotted alongside the ray tracing code output in MATLAB to compare the two results.

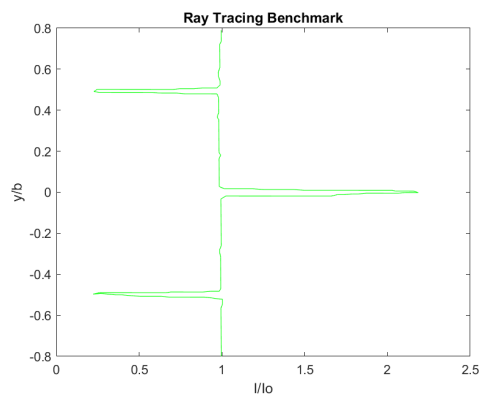


Figure 7: Ray Tracing Benchmark Data in MATLAB Extracted Using WebPlotDigitizer

The path of each ray is found by using iterative formulas. The ray tracing equation of geometric optics allows for a fast and accurate approximation of the ray path[3].

$$\frac{d}{ds}\left(f\frac{dx}{ds}\right) = \nabla f.$$

Equation 5

In order to simulate the ray path, the equation is discretized using piecewise linear approximations[3]. The position and direction of the ray are updated with every iteration of the code until the ray reaches a screen where the ray tracing stops. The position of the ray x_i is updated using the step size Δs, the refractive index f and the ray direction v_i. The ray direction is updated using the step size Δs and the gradient of the scalar field of refractive indices ∇f.

$$x_{i+1} = x_i + \frac{\Delta s}{f} v_i.$$

Equation 6

$$v_{i+1} = v_i + \Delta s \nabla f.$$

Equation 7

The step size can be modified to adjust the accuracy of the traced ray path. A larger step size will lower the accuracy of the traced ray path but will decrease computation time. In the ray tracing code, the step size was adjusted to be 1000 times smaller than the distance from the starting position of the rays to the screen. The code took around 1000 iterations for each ray to reach the screen from their starting position.

3.2 Ray Tracing Code Implementation

The code used a ray tracing function to perform ray tracing[A5]. The function required the initial position and initial direction of a ray, the higher and lower bounds of the testing area for ray tracing and the step size. Two other functions, one to calculate f and one to calculate ∇f,

was embedded inside the ray tracing function, so the ray tracing function took in the radius of the blast wave and its corresponding IOR as arguments as well.

The higher and lower bounds of the testing area for ray tracing were used to let the program know when the ray has reached the screen or has gone outside of the testing area and terminate ray tracing. For this simulation, the rays were set to travel in the positive z direction and the screen was aligned perpendicular to the z direction. The upper bound for the z direction represented the location of the screen and was usually set to a value larger than the radius of the blast wave so the rays would travel through the entire blast wave. The other upper and lower bounds were usually set slightly greater and slightly less than the radius of the blast wave.

An issue with ray tracing is that the rays do not hit the screen exactly because the ray path updates according to the step size. Originally, the code terminated once the rays past the upper boundary which is the location of the screen. A solution to this problem was to use a propagation function for the last update of each ray.

3.3 Propagation Function

The propagation function [A6] uses a matrix transformation to represent ray propagation through constant IOR over a certain distance d. Since the upper bound is larger than the radius of the blast wave and it is assumed the ambient IOR is constant, it can represent the propagation of the ray for the last update and hit the screen exactly. In the code, the parameters used to describe a ray include θ and ϕ which are the angles of elevation and azimuth, respectively, and Δx and Δy which are the offsets from the optical axis. They are illustrated in the figure below[6].

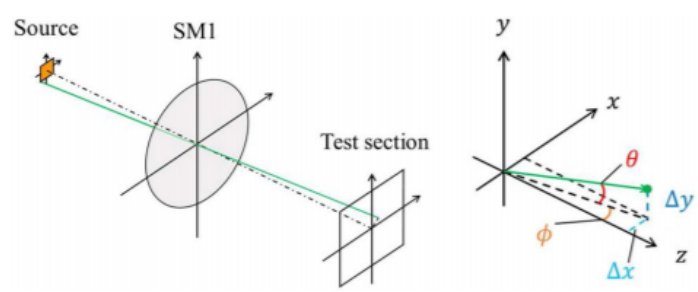


Fig. 4 Left: method for determining direction and offset at the test section initial plane caused by the extended light source. Right: geometry used to describe the offset from the optical axis, Δx and Δy, and the angle of elevation, θ, and azimuth, ϕ, of the ray

Figure 8: Ray Parameters

The ray direction vectors v_x, v_y and v_z are related to these parameters by the equations below.

$$v_x = \sin\phi\cos\theta$$

$$v_y = \sin\theta$$

$$v_z = \cos\phi\cos\theta$$

Equation 8

Equation 9

Equation 10

[6]

Using these equations and the initial position and direction of the rays before being propagated, the distance the ray travels before hitting the screen can be calculated, which is needed for the propagation transformation.

$$\begin{bmatrix} x' \\ \phi' \\ y' \\ \theta' \end{bmatrix} = \begin{bmatrix} 1 & d & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ \phi \\ y \\ \theta \end{bmatrix}.$$

Equation 11: Propagation Transformation

The propagation function includes 3 separate matrices which are used for different purposes. The function takes in a number that controls which matrix is selected, values for the other matrices, the distance d, the initial direction and position of the ray and the z coordinate of the screen[6].

3.4 IOR and IOR Gradient Functions

The IOR and IOR gradient functions [A7] [A8] take in the IOR data and the corresponding radius as inputs. It also takes in the type of blast wave, represented as 1 for a spherical blast and 2 for a cylindrical blast as well as the X, Y and Z coordinates of the desired location where the IOR or IOR gradient wants to be found. The two functions perform very similar commands until the calculation of IOR or IOR gradient which are done numerically.

First, depending on the type of blast wave, the functions find the radius using the X, Y and Z coordinates for a spherical blast or the X and Y coordinates for a cylindrical blast. The functions then check if the desired radius is within the range of radii for the IOR data. If the desired radius is greater than the largest radius for the IOR data, the functions record the two largest radii and corresponding IOR. If the desired radius is smaller than the smallest radius for the IOR data, the functions record the two smallest radii and corresponding IOR. Otherwise the functions go through the data and find the two consecutive radii, which are higher and lower than the desired radius and record the corresponding IOR.

The calculation for IOR is an interpolation of the recorded two points. The calculation for IOR gradient is slightly more complicated. First the derivative of IOR with respect to the radius is

calculated by finding the slope between the two recorded points. Since gradient is defined as the vector of partial derivatives with respect to the X, Y and Z coordinates, the gradient of IOR is the vector of partial derivatives of IOR with respect to the X, Y and Z coordinates. The chain rule can be used to find the gradient of IOR by multiplying the derivative of IOR with respect to the radius by the vector of partial derivatives of radius with respect to the X, Y and Z coordinates. The gradient of IOR for a spherical blast and a cylindrical blast are different because the spherical blast uses spherical coordinates and $r = \sqrt{x^2 + y^2 + z^2}$ while the cylindrical blast used cylindrical coordinates and $r = \sqrt{x^2 + y^2}$.

4.0 Ray Tracing Code Testing

4.1 Ray Tracing Testing Details

The ray tracing code was tested to see if it performed correctly by plotting the ray tracing code output against the benchmark. In order to do this a couple functions were added so the two results could be plotted on the same graph. In the test case given by the ray tracing benchmark, the rays were positioned in a vertical line. For replicating the test case, a function was used to initialize a uniform distribution of starting rays in a vertical line, perpendicular to the one dimensional screen in the shape of a vertical line, where the rays would hit at the end of ray tracing. The details of the function used will be given in the section Initialize Rays Function. After running each ray through the ray tracing code, the rays were simulated to hit the screen at various heights since the screen only varies in the y direction.

4.2 Application of a Pixel Array

The concept of a pixel array was used to simulate the intensity distribution of the rays on the screen. A pixel array is where there are a number of buckets which represent pixels, and each ray can be distributed into different buckets. The number of buckets a single ray is distributed into depends on the spread or range of influence of the ray. Buckets closer to where the ray hits will have more intensity than buckets closer to the edge of the ray's range of influence. More specifically there is a linear relationship between the intensity and the distance to the ray until the intensity reaches zero. The final intensity of each pixel in the array is determined using normalization of the total intensity contributed by all the rays on each bucket.

The pixel array function [A9] takes in two two dimensional arrays of starting and ending positions of the rays in coordinates. The starting and ending arrays start from the lowest height and go up to the highest height. The length each array is determined by subtracting the lowest height from the highest height. The step size or bucket size is determined by an input into the pixel array function called the pixel size factor. The step size is the height difference between two adjacent starting rays divided by the pixel size factor. The reason for this is that the number of buckets should be a multiple of the number of rays for ease of implementation.

The original start of the array is defined inside the function to be the lower bound of the starting array minus half the step size. Similarly the original end of the array is the higher bound plus half the step size. If the pixel size factor was 1, then each bucket would correspond to a single ray and the ray would be located in the center of each bucket. The degree less and degree more variable is defined inside the function to be the degree less and degree more the lower and higher bound ending rays are less and greater than the original start and original end variables. The number less and number more variables depend on the degree less and degree more variables. If the degree less and degree more variables are greater than zero, the number more and number less variables are equal to the floor of the degree less or degree more variables plus one. These represent the number of additional buckets needed to accompany the entire ending ray array. The actual start and end variables for the pixel array are defined as the original start and original end plus or minus number less or number more multiplied by the step size.

After the determination of the actual start and end variables, the total number of buckets are determined, the first and last positions of where the intensities are supposed to be calculated are determined, and arrays for the intensity and normalized intensity are initialized. The array for the intensity is 2 by the number of rays. One entry holds the y coordinate divided by b (height of the perturbed region in the benchmark IOR distribution) of the bucket in question and one holds the total intensity, the influence of all the rays. The coordinates were found by looping and adding the step size each time.

For each ray in the starting rays, the intensity distribution of the rays on the pixel array are now calculated. Since the benchmark plot requires I/I_o which is the intensity after travelling through the medium over the intensity before travelling through the medium, the intensity distribution of both the starting rays and ending rays need to be calculated. First, the range of influence of each ray is calculated by adding and subtracting the ray spread to the starting location of each ray. The lowest and highest bucket that is affected by each ray is then calculated. If the difference between the lower bound of the ray influence and the first bucket position is divisible by the step size or bucket size, then the lowest bucket will be one bucket lower than if there was still a remainder. The lowest bucket is equal to the difference between the lower bound of the ray influence minus the lower array boundary divided by the step size plus one if there is no remainder. Essentially if the range of the ray covers more than half the bucket, the bucket will be affected by the ray in the code, otherwise the bucket will not be affected. In the case where there is a remainder, the remainder is subtracted from the lower ray bound and two is added instead to calculate the lowest bucket. If the lower ray boundary affects below the first bucket position, it is immediately known that the lowest bucket is the first bucket.

A similar calculation is done to find the highest bucket, except this time using the higher bound of the ray influence in place of the lower bound of the ray influence in the calculations. Another note is that the bucket range does not have a higher bucket if there is a remainder in

the calculation now, the highest bucket remains the same whether there is a remainder or not, in order to have symmetry within the system.

After that the intensity the ray contributes to each bucket is calculated, iterating through from from the lowest bucket to the highest bucket affected calculated in the previous step. The difference parameter is used to determine how far the position of the affected bucket is from the initial location of the ray. The difference parameter is used to determine the intensity contribution of the ray to a particular bucket, because buckets farther away from the initial position of the ray are affected less. The parameter is equal to the absolute value of the position of the affected bucket minus the position of the initial ray. An accumulating sum for each bucket is stored in the plot_norm_intensities array for the accumulating intensity contributions of each ray. The intensity each ray contributes to an affected bucket is equal to one minus the difference parameter over the ray spread.

After the plot_norm_intensities array is completely populated with the contributions of all the starting rays, the normalization constant or normalization intensity I_o is calculated, the intensity of the rays before travelling through the medium. It is equal to the average intensity of all the rays affecting the buckets.

After that the same process is repeated with each ray in the ending rays to find the intensity distribution in the ending rays. This time the accumulated intensities of each final ray is recorded in the array plot_intensities. After the intensity array is completely populated with the intensity contributions of each ending ray, the total intensity of each bucket is divided by the normalization intensity I_o to get I/I_o which is required for the benchmark plot. The pixel array function returns the plot_intensities array which will be plotted in the main function.

4.3 Ray Trace Plotting Function

The main function for ray tracing testing is called plot_ray_trace [A10]. The function first initializes a few parameters that the model needs. The function sets the phi and theta of the source to zero so the source is pointing in the z direction, which is the direction of the screen. The function sets the ray pattern to 1 so the function will initialize the rays in a single line.

The function then initializes another set of parameters that can be changed by the user depending on the schlieren image desired. The function first sets the number of rays in the ray pattern. The function then sets the x and y length of the source, which is the area or window of which the image captures. Either the x or y length has to be zero since the ray pattern is set to one. If the x length is zero then the ray pattern will be a vertical line, if the y length is zero then the ray pattern will be a horizontal line. The function then sets the center of the source which should be at coordinates “0, 0, 0” but can be changed. The function also sets the higher and lower bounds of the ray tracing. The ray pattern direction (x or y depending on the chosen source length) coordinate of the higher bound should be at least one significant digit higher than

source length so ray tracing has some room to go outside the testing area before the program terminates. Similarly, the ray pattern direction coordinate of the lower bound should be at least one significant digit less than the source length. The z coordinate of the higher bound represents the location of the screen and can be set at an arbitrary value (no larger than 1000 times the step size due to computation speed) and the program should terminate ray tracing when it reaches the boundary. All the other coordinates should be set to positive or negative one depending if it is a higher or lower bound.

The function then initializes the step size which should be around 1000 times less than the distance to the screen to have a low computation time. The function then initializes a few constants needed in the program which include n_o, b, ε and Δz, which are used to simulate the benchmark. b and Δz can be set arbitrary as long as the program gives good results. n_o should be set to an arbitrary value close to 1 to simulate the IOR in air. ε is set to 0.0001 as suggested by the paper.

After setting parameters, the program sets the initial direction based on the values of phi and theta and equations 8, 9 and 10. The program initializes the rays for ray tracing using the parameters set and the set ray pattern function. It also initializes a two dimensional matrix to hold the final ray coordinates.

The program then commences ray tracing. The program takes the location of the initial rays from the initial ray matrix to be the starting location for the ray tracing function. The endpoint for ray tracing is found by taking the last entry of the location for ray tracing and it is recorded in the final ray location matrix.

The program then inputs the initial and final ray matrices in the pixel array function to convert the data into units that the benchmark data uses for plotting. The final steps the program does is to create plots with the ray tracing data and the benchmark data on the same figure.

4.4 Ray Tracing Testing Results

The results of ray tracing testing [A11] is shown below. Overall, the results of ray tracing corresponds pretty well with the benchmark plot. The peaks from ray tracing shown on the plot occur at the same place as the benchmark. However, the peaks from ray tracing are significantly shorter and thinner than the benchmark.

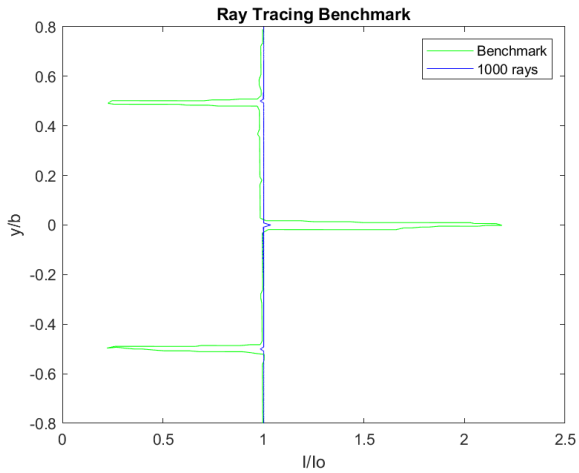


Figure 9: Ray Tracing with 1000 rays (Ray Spread = 0.05)

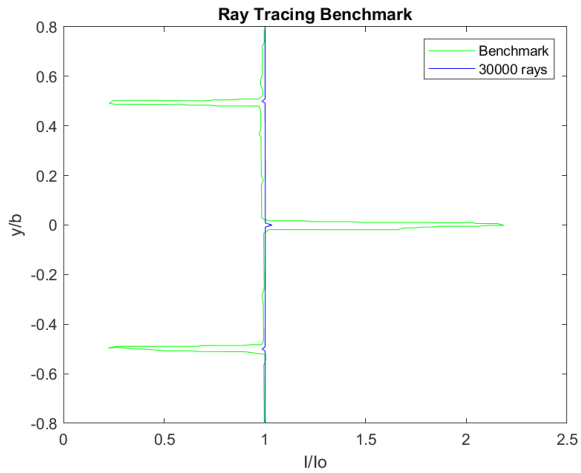


Figure 10: Ray Tracing with 30000 rays

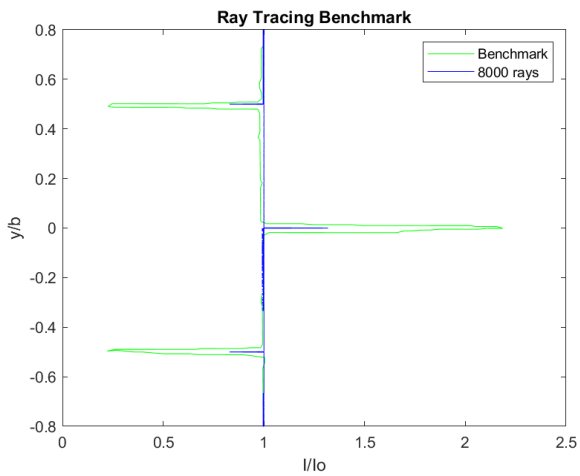


Figure 11: Ray Tracing with 0.005 Ray Spread (8000 rays)

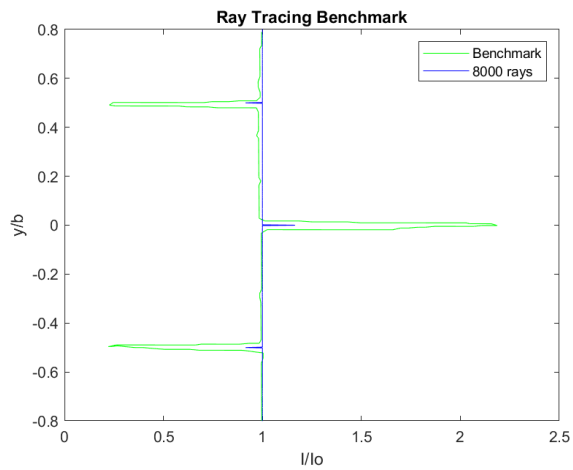


Figure 12: Ray Tracing with 0.01 Ray Spread

The results of ray tracing does not change depending on the number of rays unless the number of rays used are very low. With a ray spread of 0.05, the minimum number of rays required to create the plot is around 600. Otherwise the noise is too great to see plot details. This is different from what is reported by the paper, which saw the ray tracing converge at 6000 rays. The results of ray tracing changes depending on the ray spread however. The lower the value of the ray spread, the higher and thinner the peaks are.

Since the concept of intensity was not defined in the paper, it is possible that a different definition of intensity was used when creating the benchmark. The pixel array method may not have been used in the benchmark case. For the results of ray tracing testing, it was determined that there was a strong correlation between the ray tracing results and the benchmark results. It was deemed satisfactory to proceed to the next step for creating computational schlieren images.

5.0 Code Used For Computational Schlieren Imaging

5.1 Initialize Rays Function

The ray initialization function [A12] was used to create a uniform distribution of rays in either a line or a square. The function took in a value 1 or 2 to switch between creating a line or square ray distribution. The function also took in the number of rays, the length of the distribution in the x direction, the length of the distribution in the y direction, and the center of the distribution as arguments.

For the 1 dimensional ray distribution, the rays could either line up in the x direction (length of y = 0) or the y direction (length of x = 0). The function first finds the distance between adjacent rays by taking the non-zero length and dividing it by the number of rays minus 1. The function then finds the location of the first ray by finding the center and subtracting half of the non-zero length. The function then initializes an array of size 3 by the number of rays to hold the coordinates of each ray. The first three values in the array are set to the location of the first ray. The function uses a loop to keep adding the distance between rays for the appropriate coordinate to the location of the previous ray until the array is completely filled with the coordinates of all the rays.

For the 2 dimensional ray distribution, the function first checks to see if the x and y lengths of the ray distribution are equal, because the function can only make a square distribution. The function rounds the number of rays to the nearest square number in order to make a perfectly square ray distribution. The function does this by rounding the square root of the number of rays and squaring the value. Then the function finds the distance between each ray taking the length of x (length of y is the same) and dividing it by the number of rays minus 1. The function initializes a three dimensional array which is the number of rays in the x direction by the number of rays in the y direction by 3, to hold the coordinates of all the rays. The function then finds the location of the first ray by subtracting half of the x length and half of the y length from the center of the distribution and sets the value as the first entry in the array.

The function then uses a double loop which each loops the same number of times as the number of rays on each side of the distribution to fill the array. The function does nothing if both loops are 1 since the first entry is already filled. The function first populates the array in the y direction by adding the distance between rays in the y direction to the previous entry. When the function reaches the end of the array in the y direction, the inner loop resets back to 1. The function then adds an entry in the x direction at the starting y coordinate by adding the distance between rays in the x direction. The function would then continue populating the array in the y direction at the new x coordinate. The process continues until the array is completely populated and both loops terminate.

5.2 Knife Edge Functions

To create real schlieren images, knife edges are used to create the black and white colour distribution of the images. The knife edge blocks a portion of the light that hits the camera causing areas of an image to be a particular colour. The amount of light that is blocked depends on how much the light rays bend which is related to the IOR of the medium the rays travel through. However, the colour distribution can change when using a different knife edge, ie. the areas of the image that are different shades of black and white are different, even if what the image is showing is exactly the same.

For computational schlieren the knife edges are simulated to provide a particular colour distribution for each knife edge. The knife edges can be represented by simple mathematical equations that depend on the distance between each ray's starting and ending position in the xy plane. A black and white image in MATLAB can be represented by a matrix of intensity values from zero to one, a zero being pure black and a one being pure white with different shades of grey in between. Each entry in the matrix represents the intensity of a single pixel which is in the same spot as the image. Remember that each light ray that is ray traced is mapped to an individual pixel in the image. The knife edge formula maps a ray's x and y displacement to a single value between zero and one. That value is the intensity for that pixel and corresponding ray.

For the blast wave project, four different types of knife edges were used. The four knife edges were a horizontal knife edge, a vertical knife edge, a circular cutoff, and an inverted circular cutoff. The knife edge equations use the vector e which is the displacement between the initial ray position and final ray position when the ray hits the screen. e_x is the displacement in the x direction of the camera/image which is equivalent to the displacement in x direction of the coordinate system since the x-axis of the camera/image is aligned with the x-axis of the coordinate system. e_y is the displacement in the y direction of the camera/image which is equivalent to the displacement in y direction of the coordinate system with similar logic [3].

$$\vec{e} = (\vec{d} - \vec{d}_o),$$

Equation 12

$$e_x = \vec{e} \cdot \vec{camera}_x,$$

Equation 13

$$e_y = \vec{e} \cdot \vec{camera}_y,$$

Equation 14

The horizontal knife edge intensity depends on e_x while the vertical knife edge depends on e_y. The circular cutoff just takes into account the magnitude of the vector e which incorporates both x and y displacement. Note that the constant k represents a constant for which the equation makes sense. Since the intensity should be between zero and one, the value of k can be 0.5/maximum magnitude ray displacement for the first two equations. That

way the intensity will never surpass zero or one. For the last equation, k can be 1/maximum magnitude ray displacement for the same reasons[3].

$$I = 0.5 - e_y * k \qquad \text{(horizontal knife-edge)}, \qquad \text{Equation 15}$$

$$I = 0.5 - e_x * k \qquad \text{(vertical knife-edge)}, \qquad \text{Equation 16}$$

$$I = 1 - |\vec{e}| * k \qquad \text{(circular cutoff)}, \qquad \text{Equation 17}$$

The code for the knife edge functions [A13] are very similar. The function takes in two three dimensional matrices, one being the locations of the initial rays and one being the locations of the final rays. The first matrix is the same matrix created from the initialization of the rays. The second matrix is the same matrix after the results of ray tracing. The three entries in every row and column combination are the final coordinates of each corresponding ray. The initial ray matrix is subtracted from the final ray matrix to find the displacement. The displacement for a particular coordinate is recorded which is determined by the knife edge function. For example for the horizontal knife edge function, only the y displacement is taken. The function then searches each column and row to find the largest magnitude y displacement. From there, the value of k is found, as explained in the last section. The function then initializes a two dimensional matrix that maps to each pixel in the image to represent an array of intensities for each pixel. The value of each entry of the array depends on the displacement of the recorded coordinate for the corresponding ray and the formula for the particular knife edge.

When creating the schlieren images, the equations for the knife edges were sometimes modified so the images would look like the experimental images. Sometimes the range of intensities were adjusted so the largest displacements would not be pure black or white. In order to do this, the k value was decreased. For example, for a horizontal knife edge the k value could be 0.3/maximum magnitude displacement, in order to have the intensity range from 0.2 - 0.8 instead of 0 - 1. Sometimes the whiter areas of the image needed to be darker and the darker areas needed to be darker as well. In that case, the constant value in the equation was changed, from 0.5 to 0.4 for the horizontal knife edge. However, the intensity values still need to range from 0 - 1.

5.3 Main Function For Creating Schlieren Images

The main function [A14] for creating the schlieren image is called create_schlieren. The function first initializes a few parameters that the model needs. The function sets the phi and theta of the source to zero so the source is pointing in the z direction, which is the direction of the screen. The function sets the ray pattern to 2 so the function will initialize the rays in a square. The function also sets the number of rays as 512*512 and the number of rays in the x

and y directions as 512 in order to create a 512x512 pixel image. The parameters listed above stay constant and should not be changed.

The function then initializes another set of parameters that can be changed by the user depending on the schlieren image desired. The function first sets the x and y size of the source, which is the area or window of which the image captures. The blast waves simulated are spherical or cylindrical in nature so the testing area is a cube at coordinates 0, 0, 0 in order to preserve symmetry. Each side of the cube is slightly larger than the diameter of the blast wave so the entire blast wave should fit in the testing area. The testing screen represents one side of the cube, perpendicular to the z-axis. The function then sets the center of the source which should be at coordinates “0, 0, half the window size”. The function also sets the higher and lower bounds of the ray tracing. The x and y coordinates of the higher bounds should be one significant digit higher than half the window size so ray tracing has some room to go outside the testing area before the program terminates. The z coordinate of the higher bounds is set at half the window size however, because the program should terminate ray tracing when it reaches the screen at the end of the testing area. The lower bound coordinates are all set to the negative of one significant digit higher than half the window size. The function then initializes the size which is the length of the image window and the step size which is set at size/1000. The step size allows for ray tracing to iterate roughly 1000 times before terminating in order to reduce the computation time of the ray tracing program. The function then initializes t, which tells the function what radius and index of refraction data to use for ray tracing, effectively choosing what blastwave to make an image of. Finally, the function initializes a type parameter which indicates whether it is a spherical blast wave or a cylindrical blast wave.

After setting parameters, the program calls the data selection function [A15] which loads the radius and density data required for the blast wave. Then the program sets the initial direction based on the values of phi and theta and equations 8, 9 and 10. The program initializes the rays for ray tracing using the parameters set and the set ray pattern function. It also initializes a three dimensional matrix to hold the final ray locations.

The program then commences ray tracing. The program takes the location of the initial rays from the initial ray matrix to be the starting location for the ray tracing function. The endpoint for ray tracing is found by taking the last entry of the location for ray tracing and it is recorded in the final ray location matrix. The program uses two loops to go through every row and column of the initial ray matrix to do ray tracing on every ray. Then an intensity matrix is created using a knife edge function and by inputting the initial and final ray location arrays into the function. Finally an image is made by writing the intensity matrix to a file as an image and reading the image.

6.0 Creating Schlieren Images

A set of schlieren images for both the spherical and cylindrical blast waves were created using the schlieren imaging program. The blast wave data came from research papers from

outside sources. The spherical blast wave data used the density distributions given by Yan's paper[1] and the cylindrical blast wave data used the density distributions given by Plooster's paper[4].

Images were generated for all four spherical blast waves and the four smallest cylindrical blast waves from the data set. Three different knife edges were used on each blast wave, a vertical knife edge, a horizontal knife edge, and a circular cutoff, making a total of 24 generated images. Each image took around 30 minutes to an hour to generate. A couple of generated schlieren images are shown below. A complete set of images is shown in the appendix [A16].

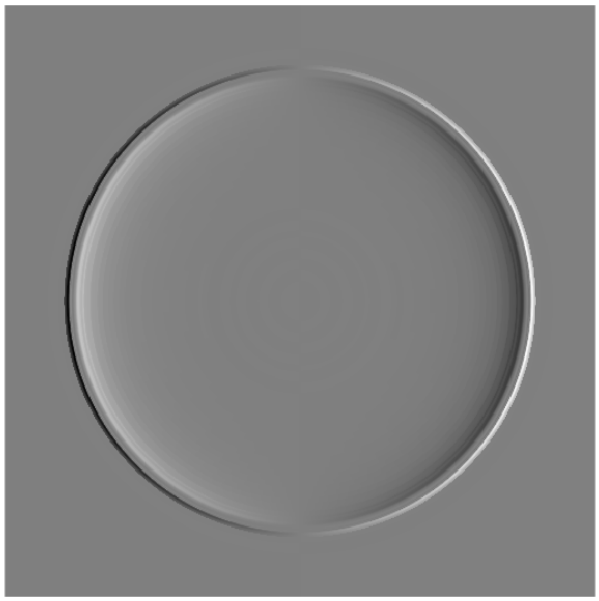


Figure 13: Cylindrical Blast Horizontal Knife Edge

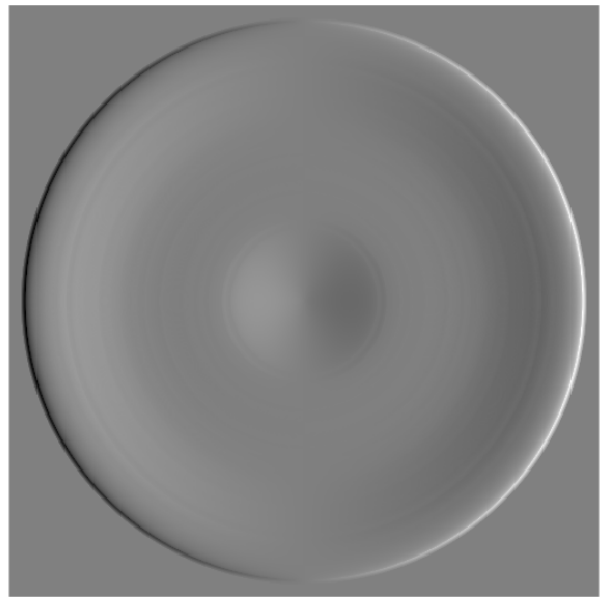


Figure 14: Spherical Blast Horizontal Knife Edge

After viewing the schlieren images, it seemed that the experimental schlieren could be replicated by changing the knife edge equation parameters. After some experimentation with the knife edge equation parameters, the experimental schlieren images were replicated as well as possible. The spherical blast wave was replicated using an inverse circular cutoff and the following knife edge equation: $I = 0.3 + 0.4 / d \times \sqrt{x^2 + y^2}$. d represents the maximum displacement magnitude and x and y represent the x and y displacements of the analyzed ray. The cylindrical blast wave was replicated using a circular cutoff and the following knife edge equation: $I = 0.7 + 0.7 / d \times \sqrt{x^2 + y^2}$.

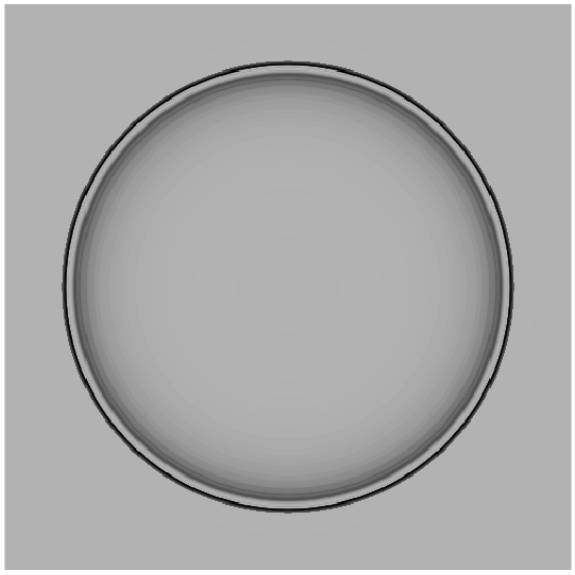


Figure 15: Cylindrical Blast Circular Cutoff

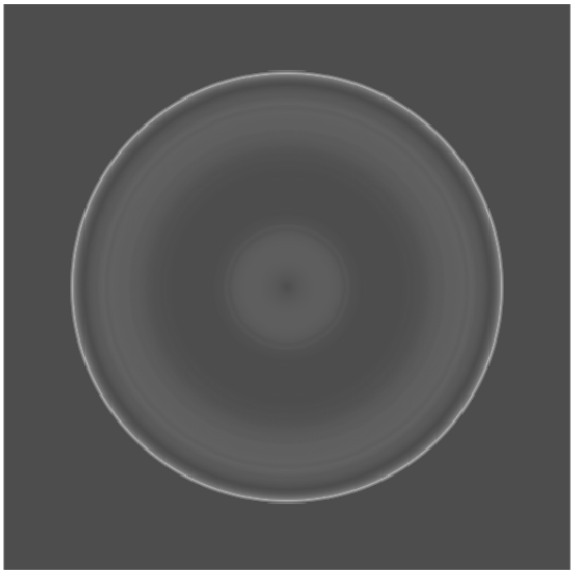


Figure 16: Spherical Blast Inverse Circular Cutoff

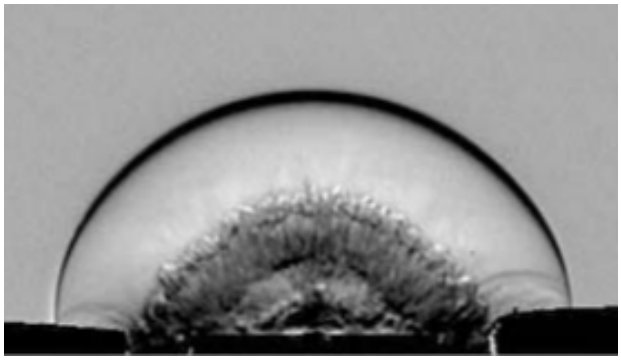


Figure 17: Experimental Cylindrical Blast Schlieren

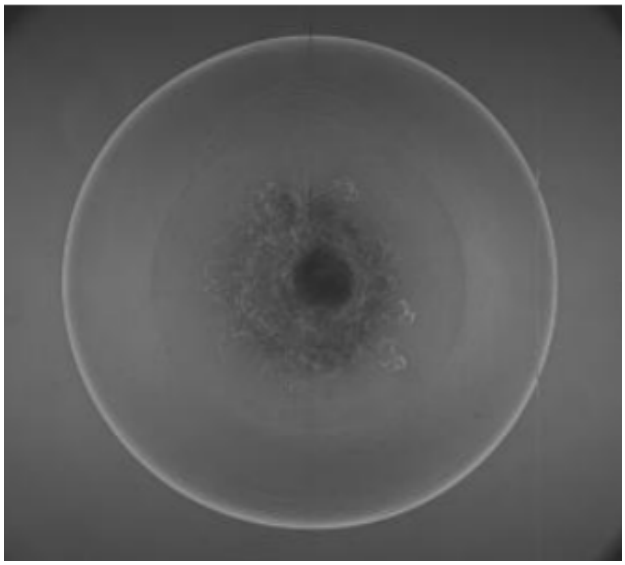


Figure 18: Experimental Spherical Blast Schlieren

7.0 Plotting Gradient of Density and Image Intensity Distribution

7.1 Plotting Gradient of Density and Image Intensity Distribution Details

In order to ensure that the ray tracing and schlieren images are accurate, the gradient of density and the image intensity distributions were plotted together. If the ray tracing and schlieren imaging is accurate, then the cylindrical blast wave density gradient should be the same as the image intensity distribution and the spherical blast wave density gradient should not be the same as the image intensity distribution because of three dimensional effects.

A new program was made that used the spherical and cylindrical density data given by the two papers. Intensity data [A17] was collected from both the experimental schlieren images as well using ImageJ. The intensity profiles of the images can be found by drawing a line from the center of the blast wave to the edge of the image and clicking “Plot Profile” under one of the toolbars. The profile and intensity data can be saved as an Excel file which was copied over to MATLAB. ImageJ was used for the generated synthetic schlieren images as well, to check that the intensity profiles matched the intensity matrix.

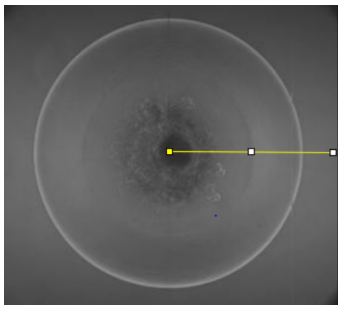


Figure 19. ImageJ Data Collection

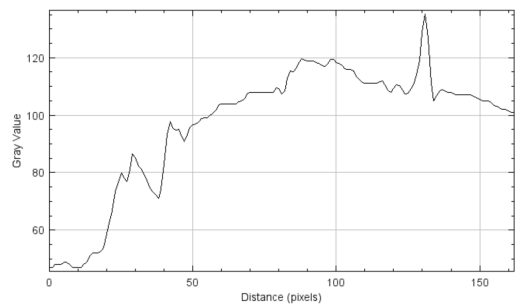


Figure 20. ImageJ Plot Profile

7.2 Plotting Program Details

The program [A18] first loads in all the density data as .mat files into MATLAB. The program then initializes a few constants including E_o, γ, p_o, a_o, R_o and ρ_o for the cylindrical blast wave. The program then determines the radius and corresponding density values for all the cylindrical blast waves by multiplying the dimensionless radius by the characteristic radius R_o to get the radius and multiplying the dimensionless density by the ambient density ρ_o to get the density. The program does the same thing for the spherical blast waves but using constants instead of variables because the characteristic radius R_o is the same for the blast waves.

The program then uses a function plot_grad_rho [A19], to plot the density gradient using the density data. The function takes in the radius and density data as inputs as well as the lowest and highest radii desired for the plot and the number of entries in the plot. First the function creates an array of equally spaced values from the lowest to the highest radii in the number of desired entries. These are the points where the density gradient will be calculated. An equal length array is initialized to hold the values of density gradient for the corresponding radius.

The method for computing density gradient is the same as explained in the IOR and IOR gradient functions section except with density. The function computes the gradient at every point in the radii array and enters the value in the density gradient array. The function returns these two arrays.

The program lists plot_grad_rho for every blast wave but only one blast wave is plotted at a time. When running the program the data plotted must be the only line uncommented. Note

that the highest radius plotted was chosen carefully beforehand by examining the plots, seeing roughly where the blast wave ended and going a bit farther to see the transition from the blast wave to ambient air. All plots used 1000 entries for the array and started at a radius of zero.

The program then finds the maximum and minimum values of the radius and density gradient which is used for normalizing the intensity data on the same plot. A figure is created and the radius and corresponding density gradient is plotted on it.

Next, the function takes the radius data for the image intensity distribution and normalizes it with the radius data for the density gradient data. The formula for normalization into a specific range is min_grad + (image - min_image)*(max_grad - min_grad)/max_image where image refers to the radius data for the image intensity distribution and grad refers to the radius data for the density gradient. The function also normalizes the intensity data with the density gradient data using the same method. The normalized data is plotted on the same figure as before. Note that only the first four cylindrical blasts had image intensity distributions, because their schlieren images were the only ones analyzed. The same normalization method was used on the experimental schlieren images. Note that the same experimental image was used for all the spherical blasts and all the cylindrical blasts since images at the specific times could not be obtained. Only one set of intensity data was used for all the spherical blasts and all the cylindrical blasts, but they were all normalized to their specific blast wave plots. Again, the experimental schlieren data was plotted on the same figure to compare with the density gradient and the generated synthetic schlieren image intensity data.

7.4 Plotting Results

The same type of plots [A20] consisting of the density gradient, the synthetic schlieren intensity distribution and the experimental schlieren intensity distribution were created for the synthetic schlieren images that used a horizontal knife edge, a circular cutoff and an inverted circular cutoff. Some plots inverted the synthetic schlieren intensity distribution or the experimental schlieren intensity distribution to show the similarities between their plots and the density gradient. It was the case for all images using the horizontal knife edge. Note that if the schlieren image used an inverted knife edge or a horizontal knife edge, the intensity distribution would be inverted compared to the intensity distribution. For the normal and inverted circular cutoff images, many of the plots had areas that were reflected at a certain height due to the absolute values in the formulas.

Overall, there was some correlation between the cylindrical blast density gradient and the image intensity. The spherical blast density gradient clearly did not match the image intensity because there was an extra curve present at the beginning of the plot for the image intensity that did not show up in the density gradient. The generated schlieren image intensity distributions were not quite close to the experimental schlieren intensity distributions, but did share a few patterns in the plots.

Due to a combination of time constraints and the results generated from the plots, it was determined that there was not enough confidence in the ray tracing code and the generated schlieren images to move forward with the project.

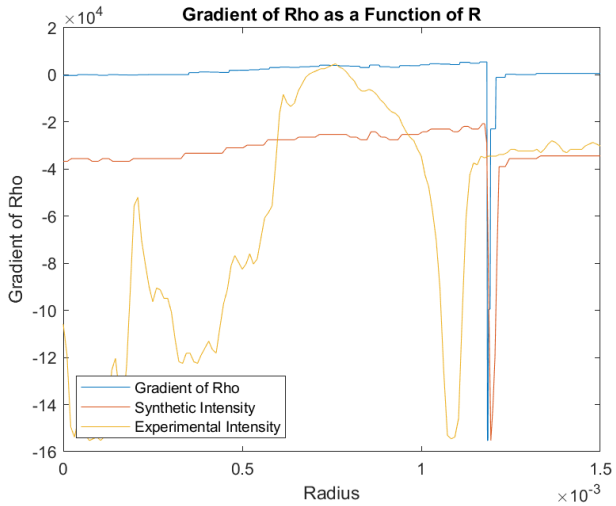


Figure 21: Cylindrical Blast Horizontal Knife Edge Plot

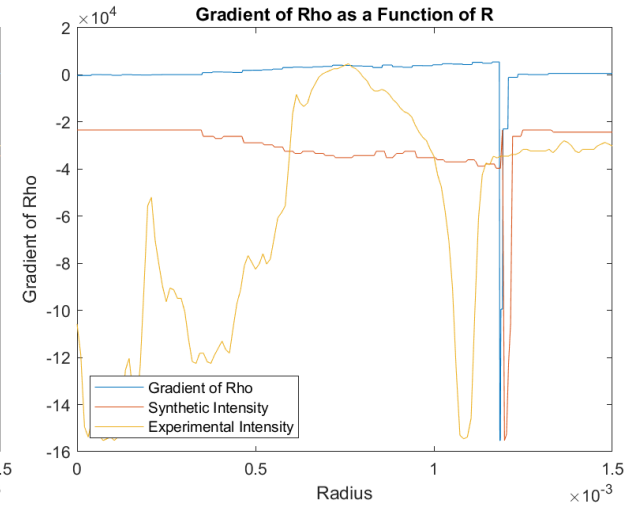


Figure 22: Cylindrical Blast Circular Cutoff Plot

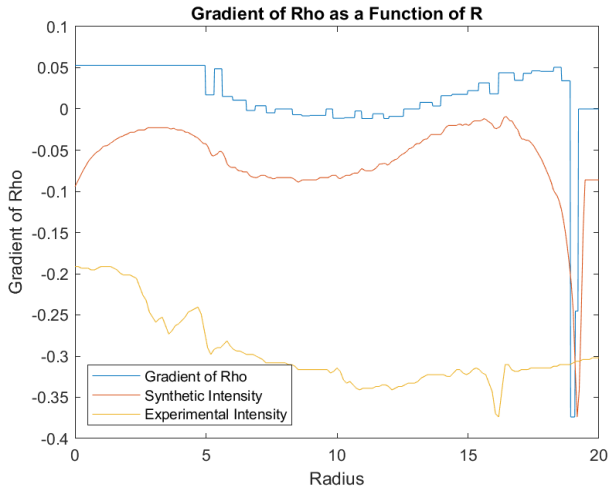


Figure 23: Spherical Blast Horizontal Knife Edge Plot

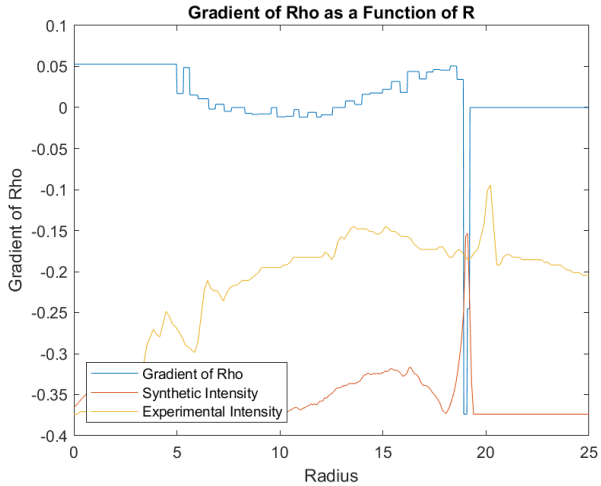


Figure 24: Spherical Blast Circular Cutoff Plot

8.0 Improving Ray Tracing and Schlieren

There were a few issues with the ray tracing method used. The method of computing the IOR and IOR gradient through interpolation and finding the derivative numerically is not accurate enough because it uses information between two consecutive data points. The larger the distance between the two data points, the greater the possible error. There are not enough data points for both the spherical and cylindrical blast wave data, about 50 - 200 points for each data set for the method to be satisfactory.

A better method of finding the IOR gradient is to use the spherical and cylindrical blast wave data and curve fitting it. This way the final schlieren image will be smoother and would

follow the general trend of the density profile. It will also reduce the error of individual data points because the curve fit takes into account the whole density profile. It is important to do a piecewise curve fit for the blast wave data because of the sharp downward slope at the edge of the blast wave and the flattening of the curve once the radius reaches the ambient air. Otherwise, a good curve cannot be found. A three piece function is needed, one for the blast wave, one for the blast wave at the shock and one for the ambient air.

WebPlotDigitizer[2] can be used to extract the three portions of each density curve. The data can then be saved in MATLAB as .mat files. There is a curve fitting add-on in MATLAB that performs curve fitting on .mat files which can be used for curve fitting each portion of the density curve. After that, the function can be manipulated using the Gladstone-Dale relation to find a function for IOR. Instead of looking for two consecutive data points that are higher and lower than a specific radius and calculating the IOR through interpolation, the IOR can be found by inputting the radius in the IOR function and getting the corresponding IOR as an output. The derivative of IOR with respect to radius can be found by inputting the radius into the derivative of the IOR function. Since the density profiles are curve fitted, the derivative will just be the derivative of the curve fitted function which can be calculated analytically. The chain rule will be used in the same way as before to find the IOR gradient.

A drawback of this method is that spherical blast data is not recorded below five dimensionless radii so a new data set would have to be found.

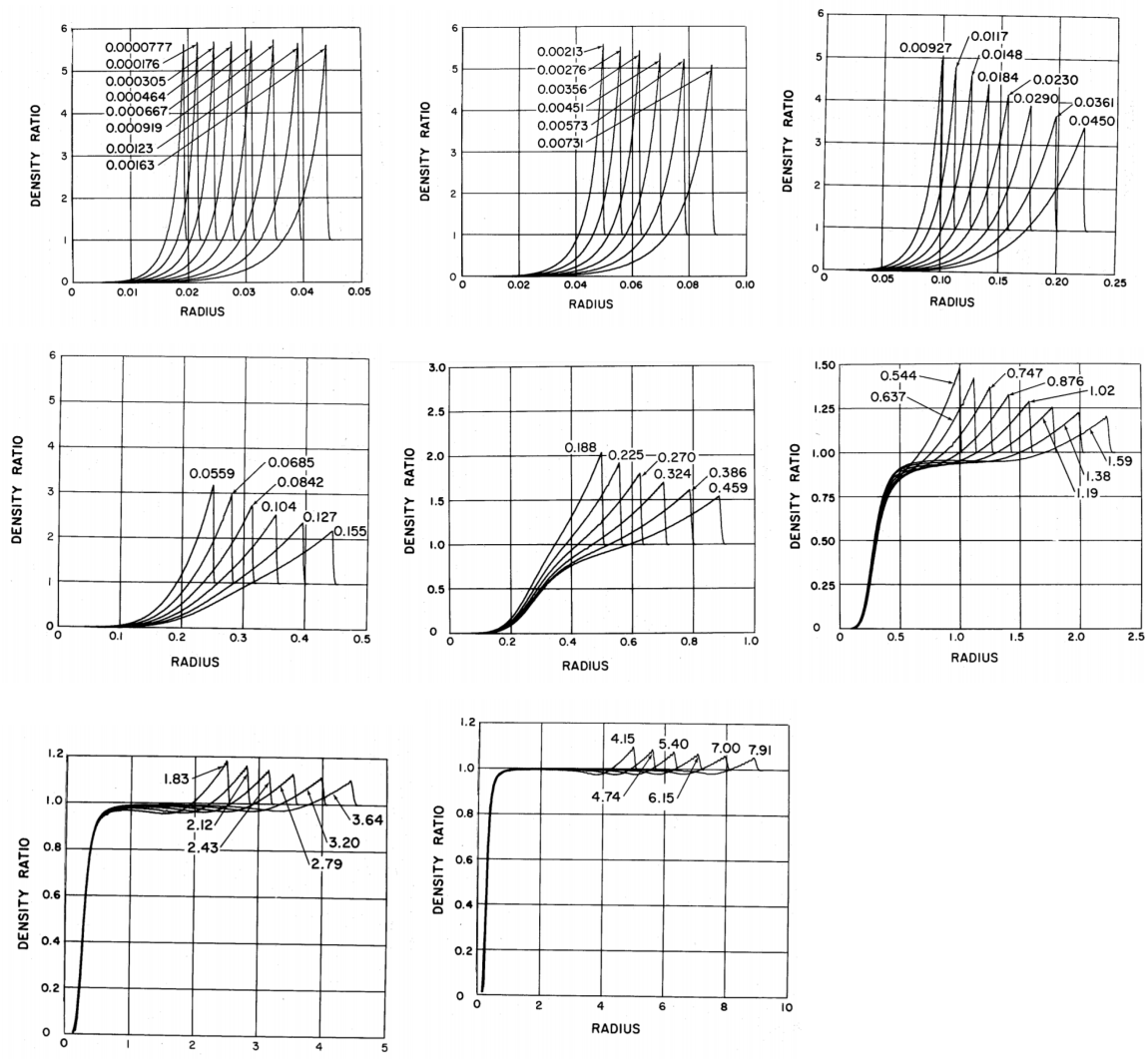
9.0 Findings

A couple of conclusions were made from the project. The ray tracing code seems to perform somewhat accurately but can be changed to increase accuracy. The next steps would be to improve on the accuracy and check the density gradient and image intensity distribution plots again using newly generated schlieren images. If the schlieren images are satisfactory, the ray tracing code can then be applied to CFD data. Five reports were created on PowerPoint to discuss findings during different stages of the project[A22].

The blast wave data should be curvefitted in MATLAB using the curve fitting tool with the data points collected from WebPlotDigitizer. The IOR and IOR gradient functions should be changed to be compatible with the curve fit data as mentioned in the areas for improvement section.

10.0 Appendix

A1. Cylindrical Blast Wave Plots



A2. Spherical Blast Wave Data

See rho_t_**x**.mat files in folder.

A3. Cylindrical Blast Wave Data

See cyl_d_**x**.mat files in folder.

A4. Ray Tracing Benchmark Data

See RayTracingBenchmark.mat file in folder.

A5. Ray Tracing Function

See new_ray_trace function in folder for benchmark ray tracing function.

See ray_trace_schlieren function in folder for schlieren ray tracing function.

A6. Propagation Function

See propagate function in folder.

A7. IOR Function

See f function in folder for benchmark IOR function.
See n_r_numerical function in folder for schlieren IOR function.

A8. IOR Gradient Function

See grad_f function in folder for benchmark IOR gradient function.
See grad_n_r_numerical function in folder for schlieren IOR gradient function.

A9. Pixel Array Function

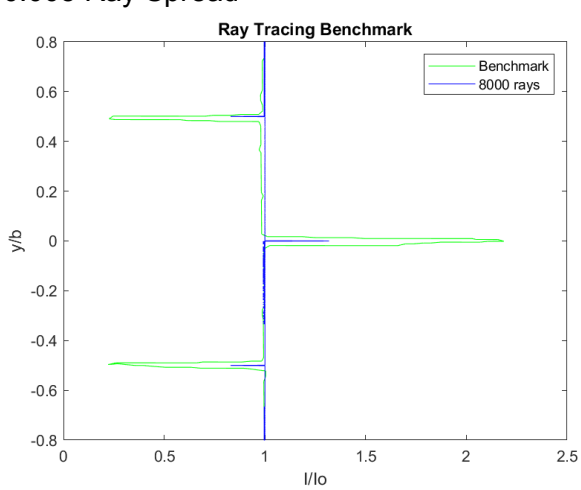
See New_Pixel_Array function in folder.

A10. Ray Tracing Plot Function

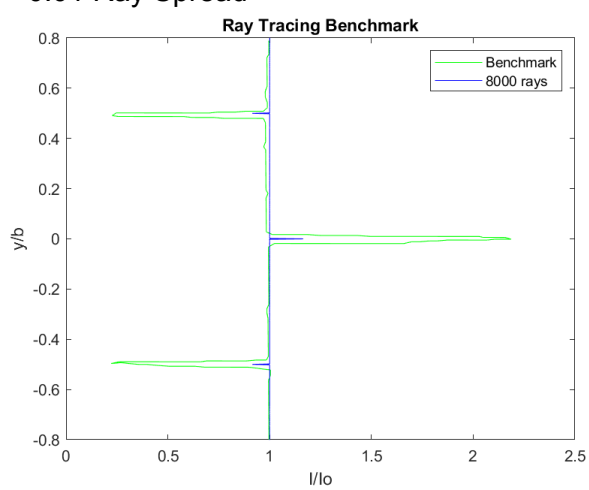
See plot_ray_trace function in folder.

A11. Ray Tracing Plots

0.005 Ray Spread

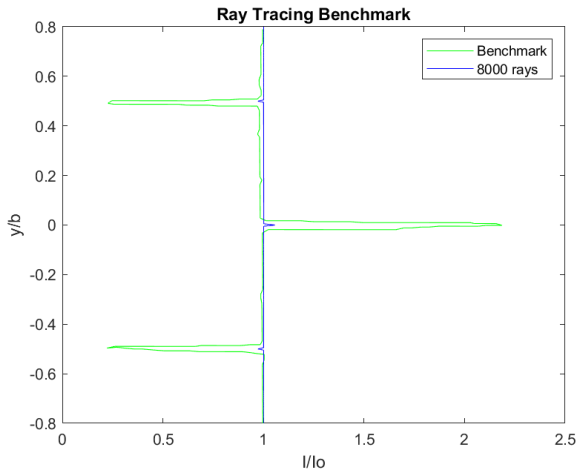


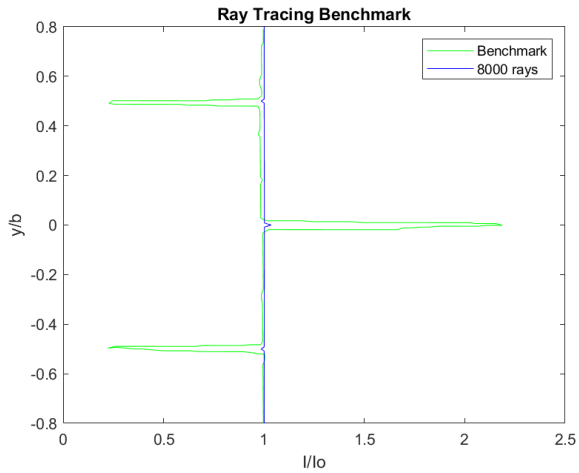
0.01 Ray Spread



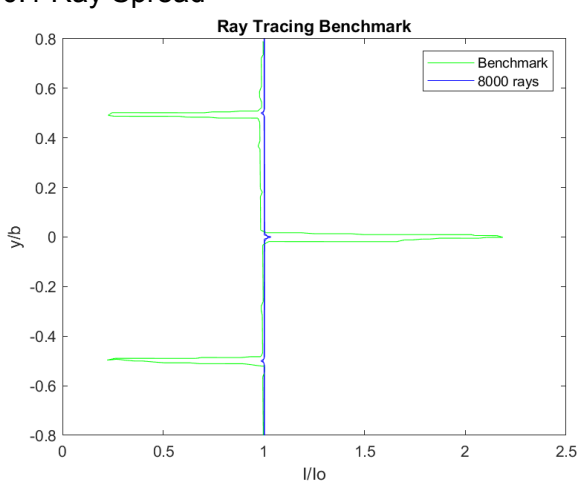
0.03 Ray Spread

0.05 Ray Spread

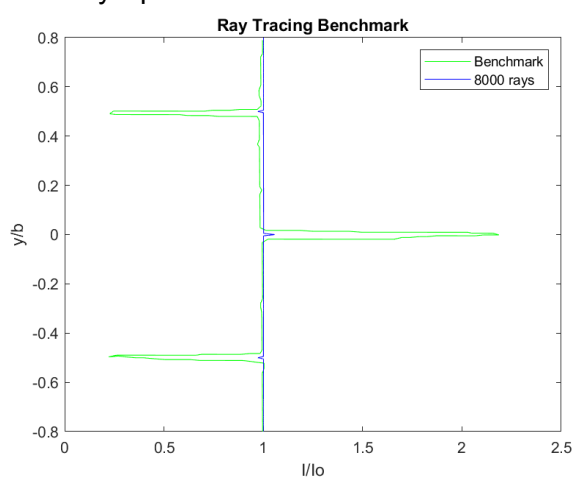




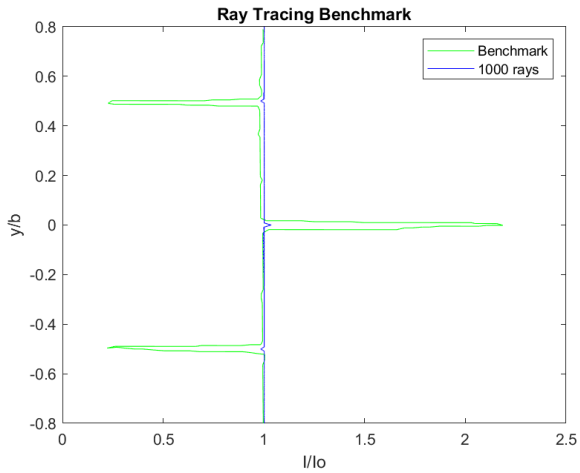
0.1 Ray Spread

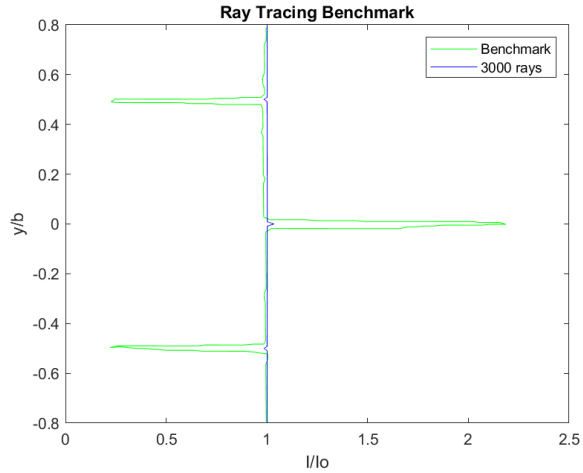


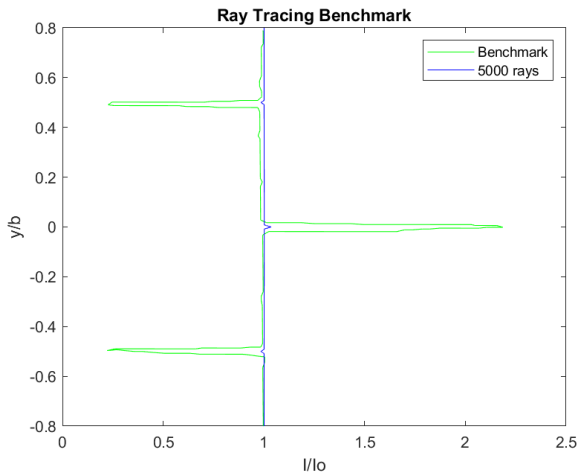
0.3 Ray Spread

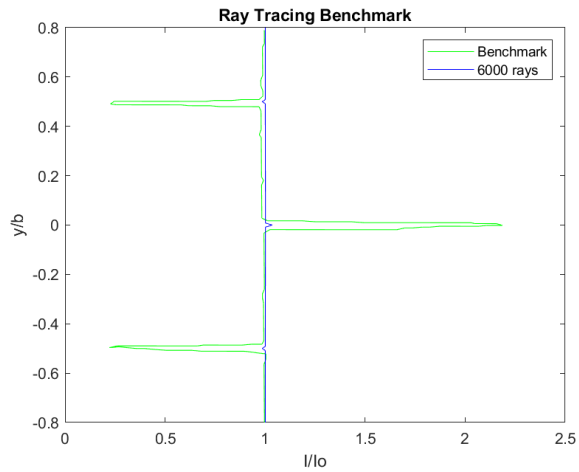


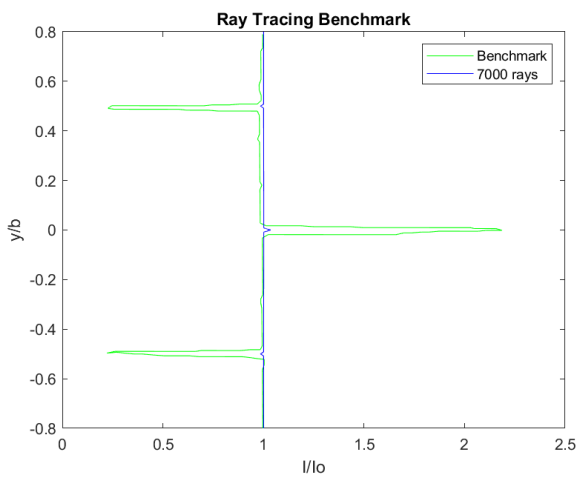
0.05 Ray Spread

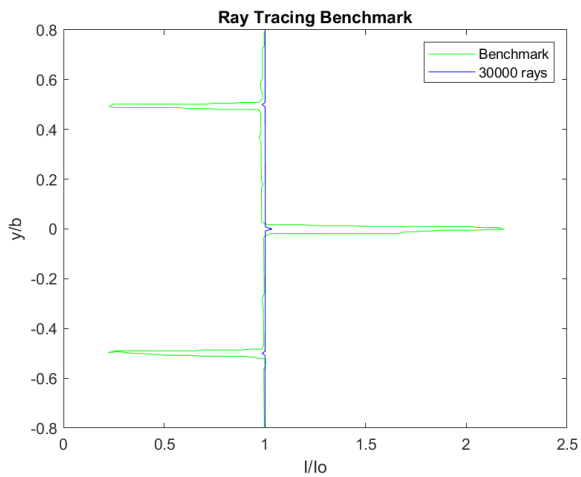












A12. Initialize Rays Function

See set_rays_pattern function in folder.

A13. Knife Edge Functions

See horizontal_knife_edge function in folder.
See vertical_knife_edge function in folder.
See circular_cutoff function in folder.
See inv_circular_cutoff function in folder.

A14. Create Schlieren Image Function

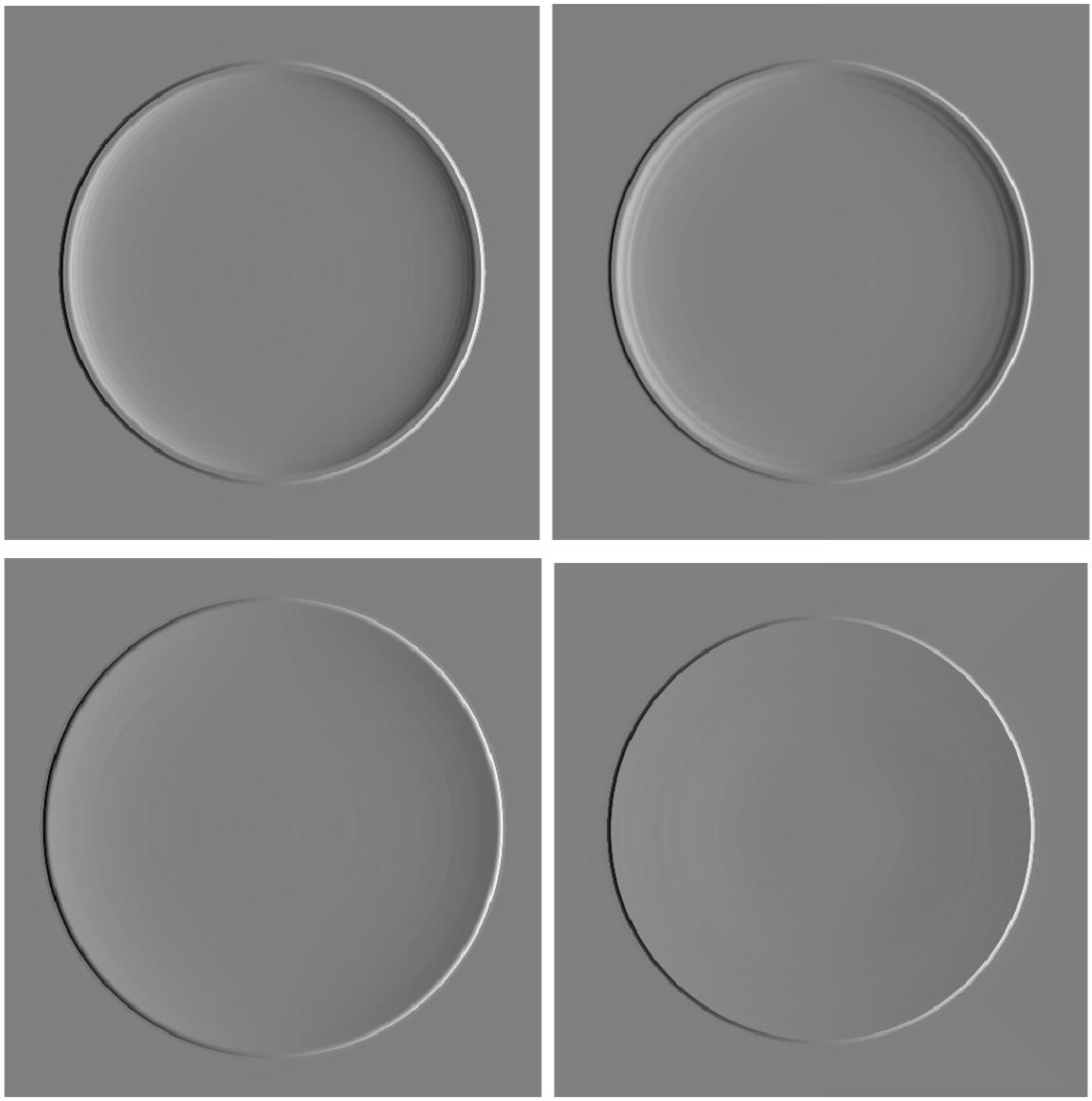
See create_schlieren function in folder.

A15. Data Selection Function

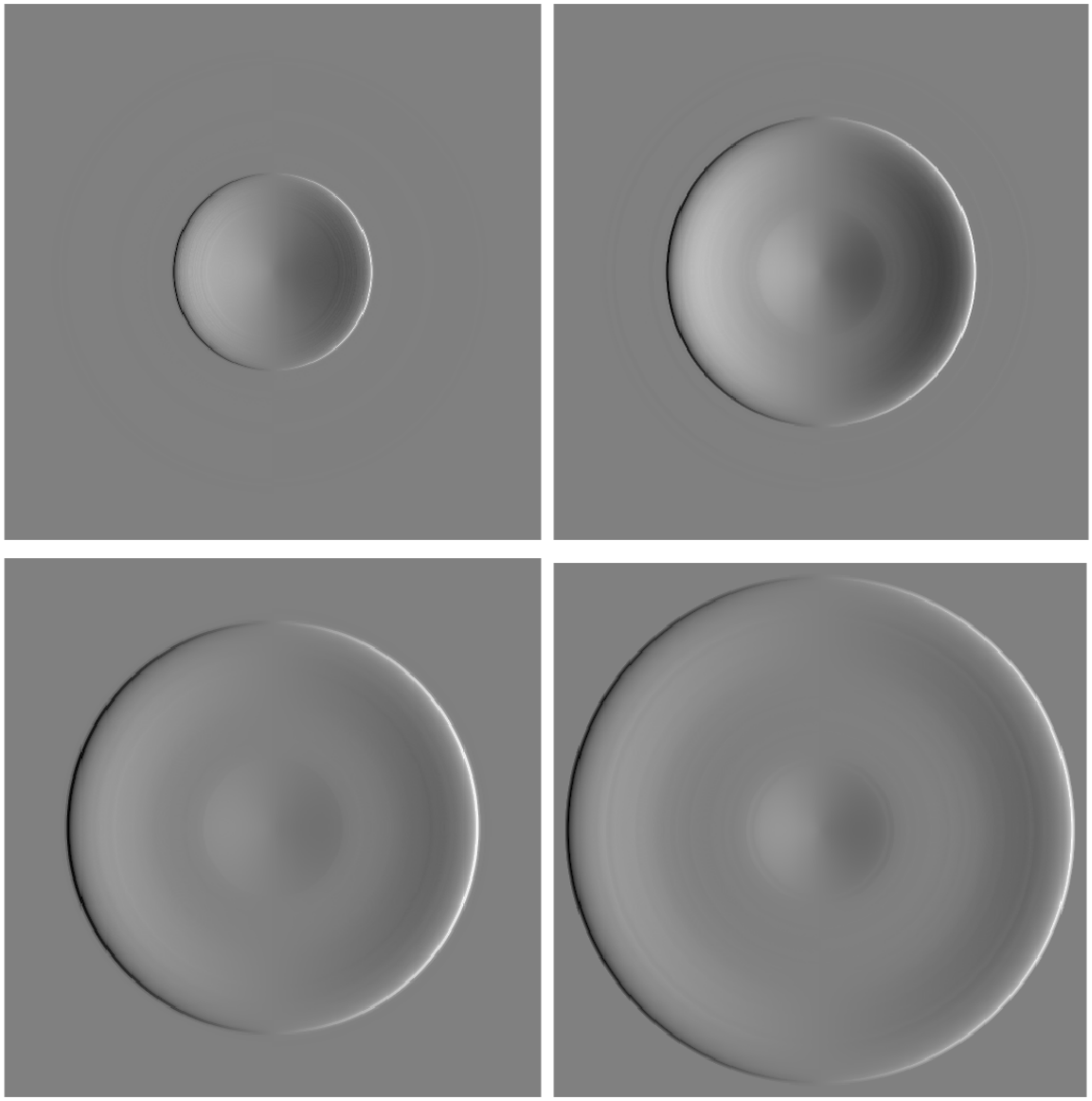
See data_selection function in folder.

A16. Generated Computational Schlieren Images

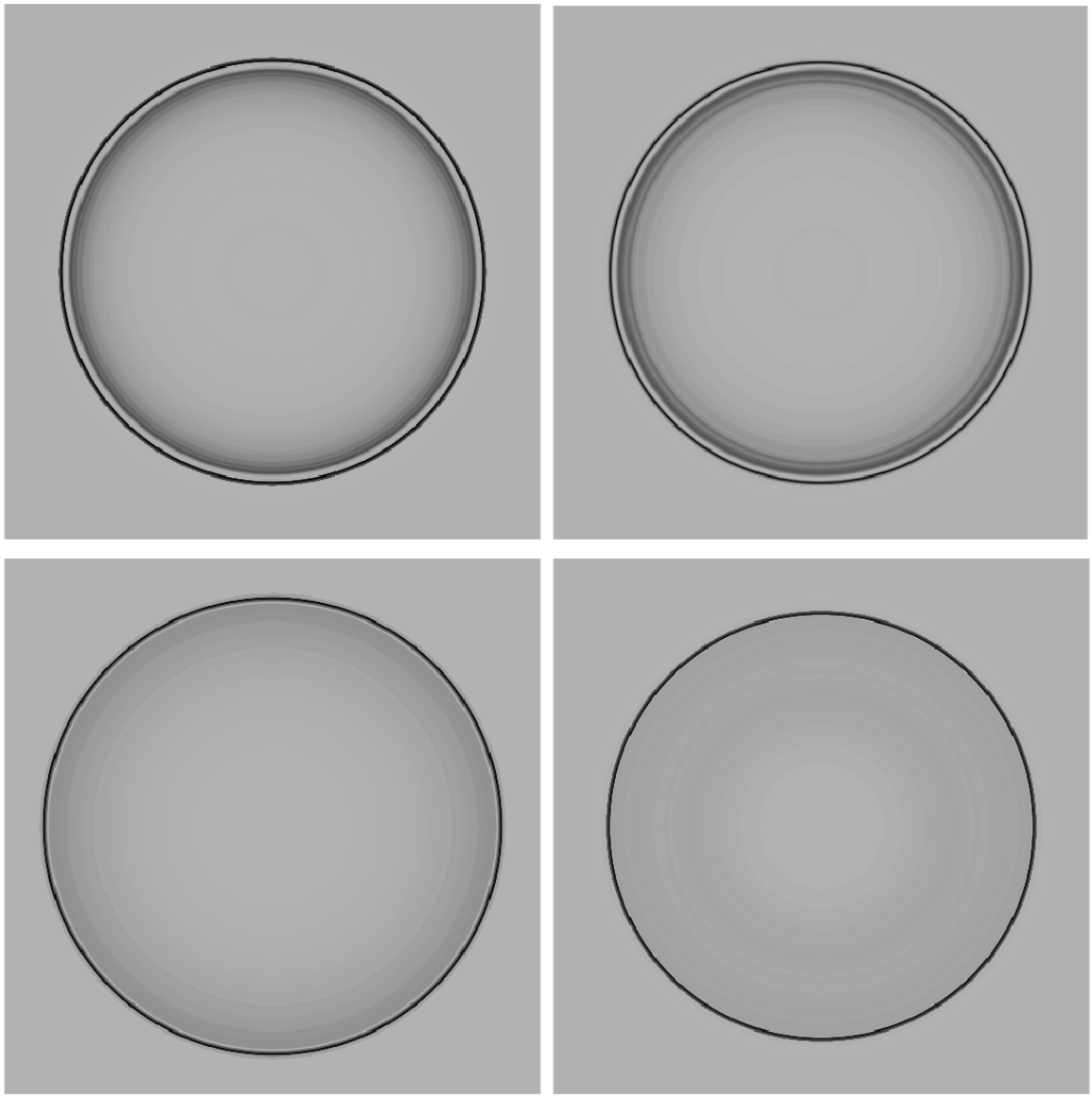
Generated Cylindrical Blast Schlieren Using Horizontal Knife Edge (Increasing Time)



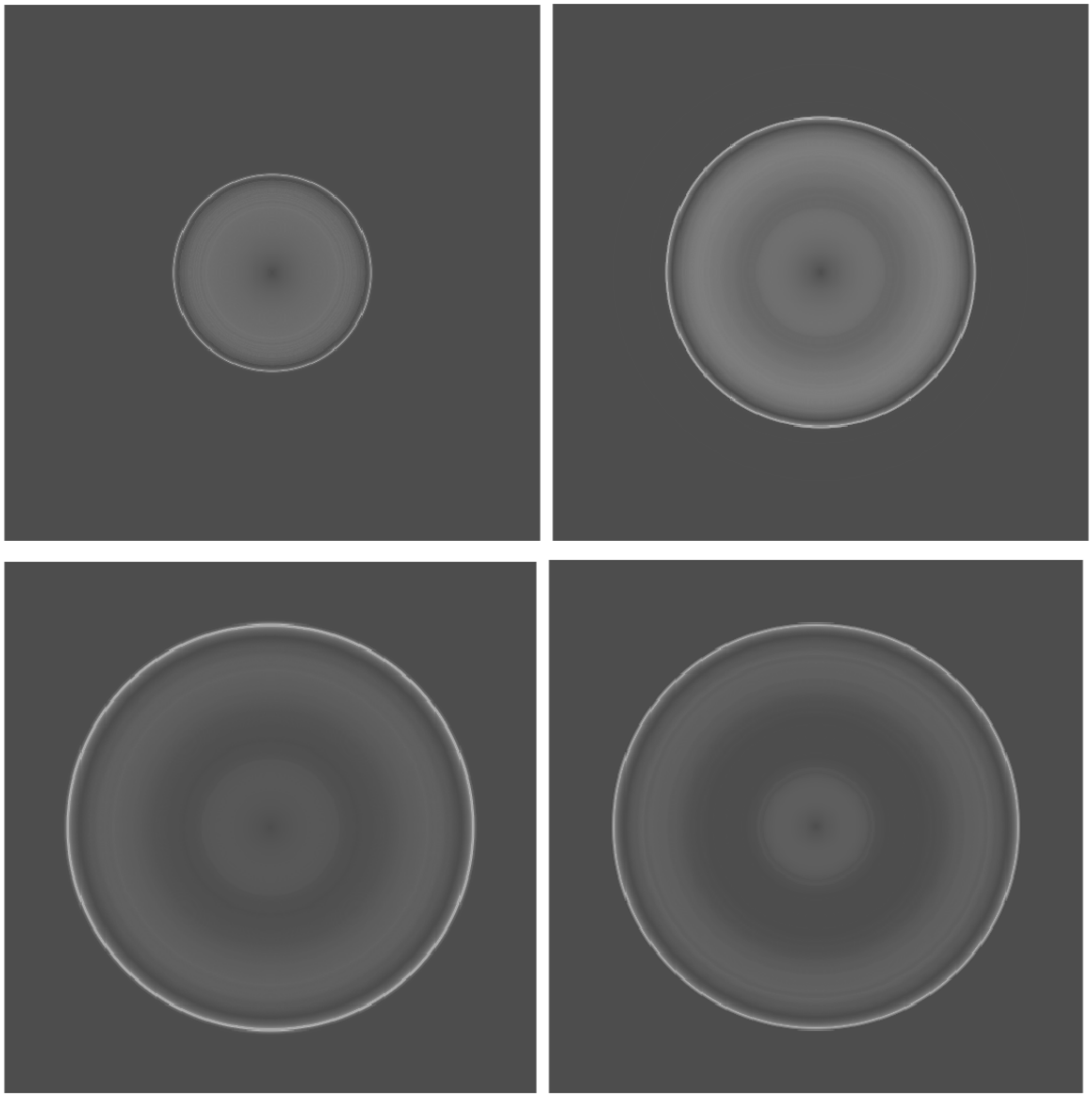
Spherical Blast Schlieren Using Horizontal Knife Edge (Times from 10 - 40 us)



Generated Cylindrical Blast Schlieren Using Adjusted Circular Cutoff (Increasing Time)



Generated Spherical Blast Schlieren Using Adjusted Inverted Circular Cutoff (Times from 10 - 40 us)



A17. Intensity Data From ImageJ in MATLAB

See intensity_data_**x** for generated circular and inverse circular cutoff image intensity data.
See intensity_data_**x**_old for generated horizontal knife edge schlieren image intensity data.
See intensity_data_cyl for cylindrical blast experimental schlieren image intensity data.
See intensity_data_sph for spherical blast experimental schlieren image intensity data.

A18. Plotting Density Gradient and Intensity Distribution Program

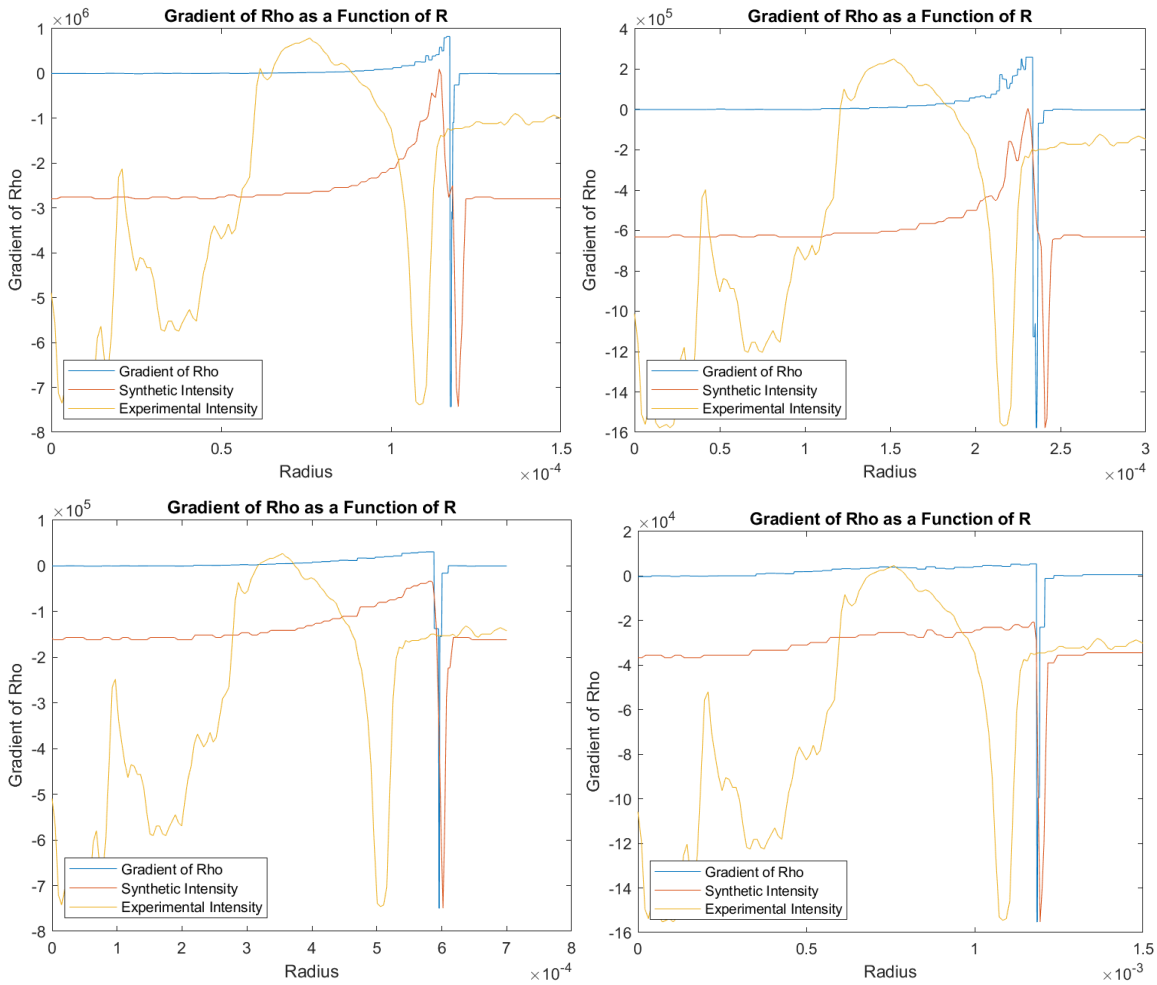
See intensity_figure function in folder

A19. Plot Density Gradient Function

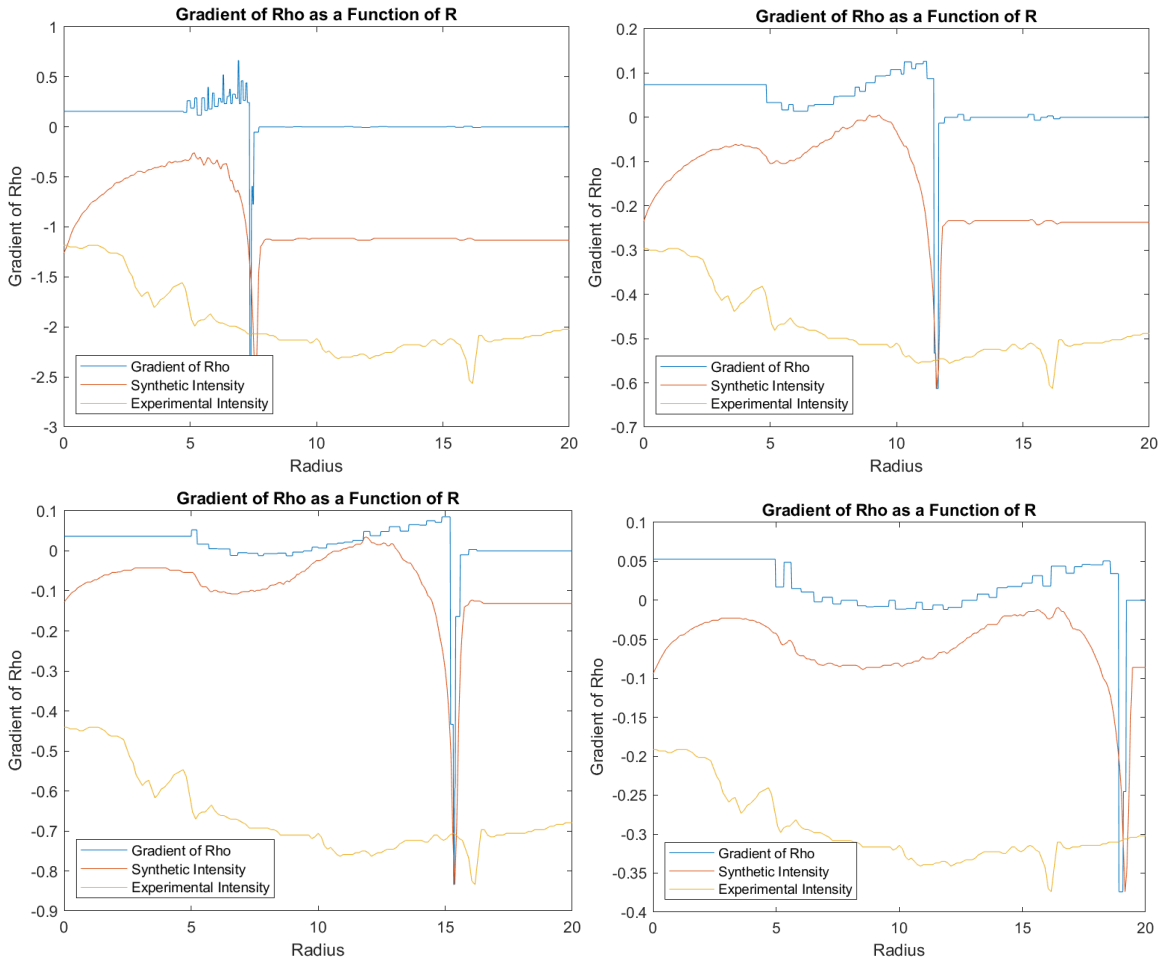
See plot_grad_rho function in folder.

A20. Density Gradient and Intensity Distribution Plots

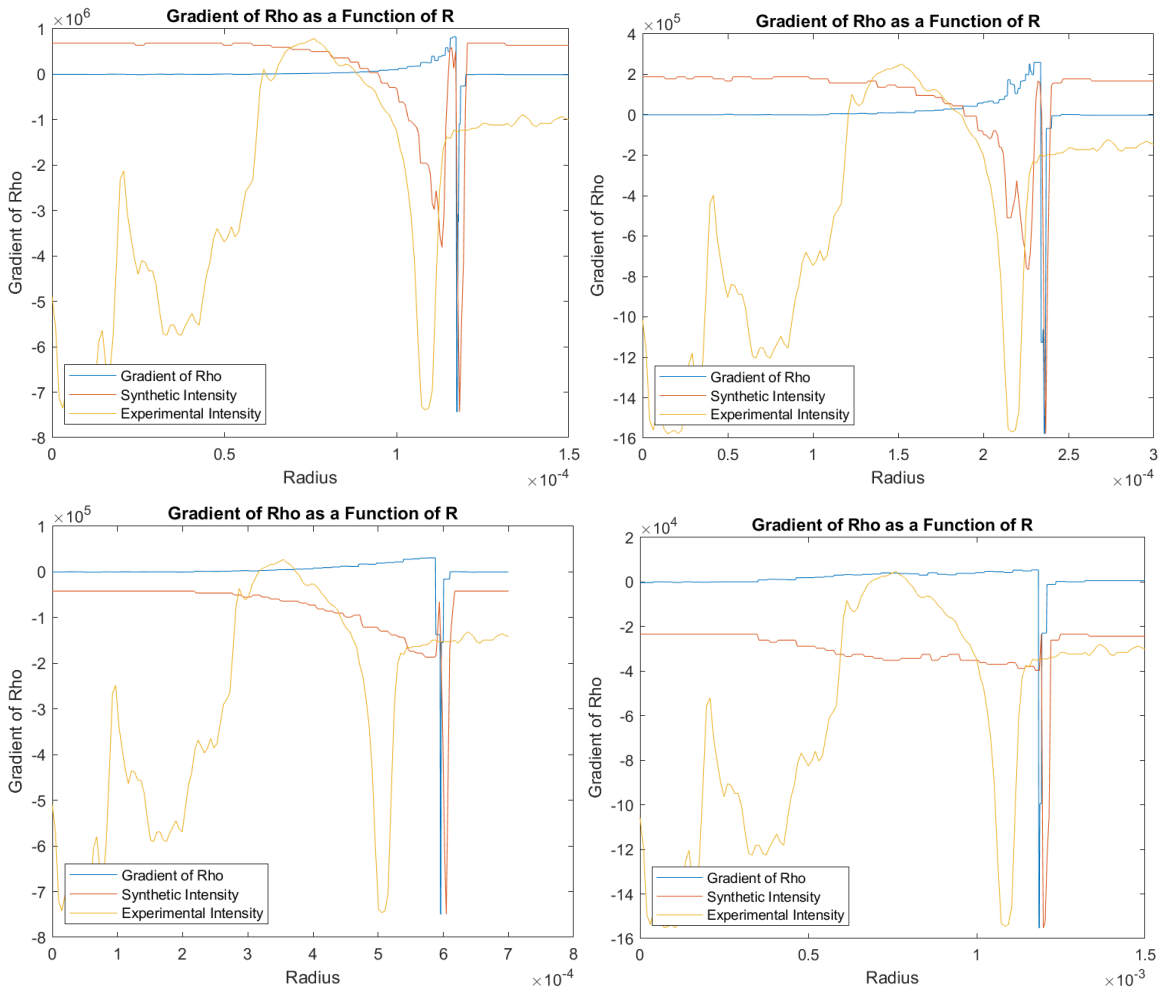
Cylindrical Blast Horizontal Knife Edge Plots (Increasing Times)



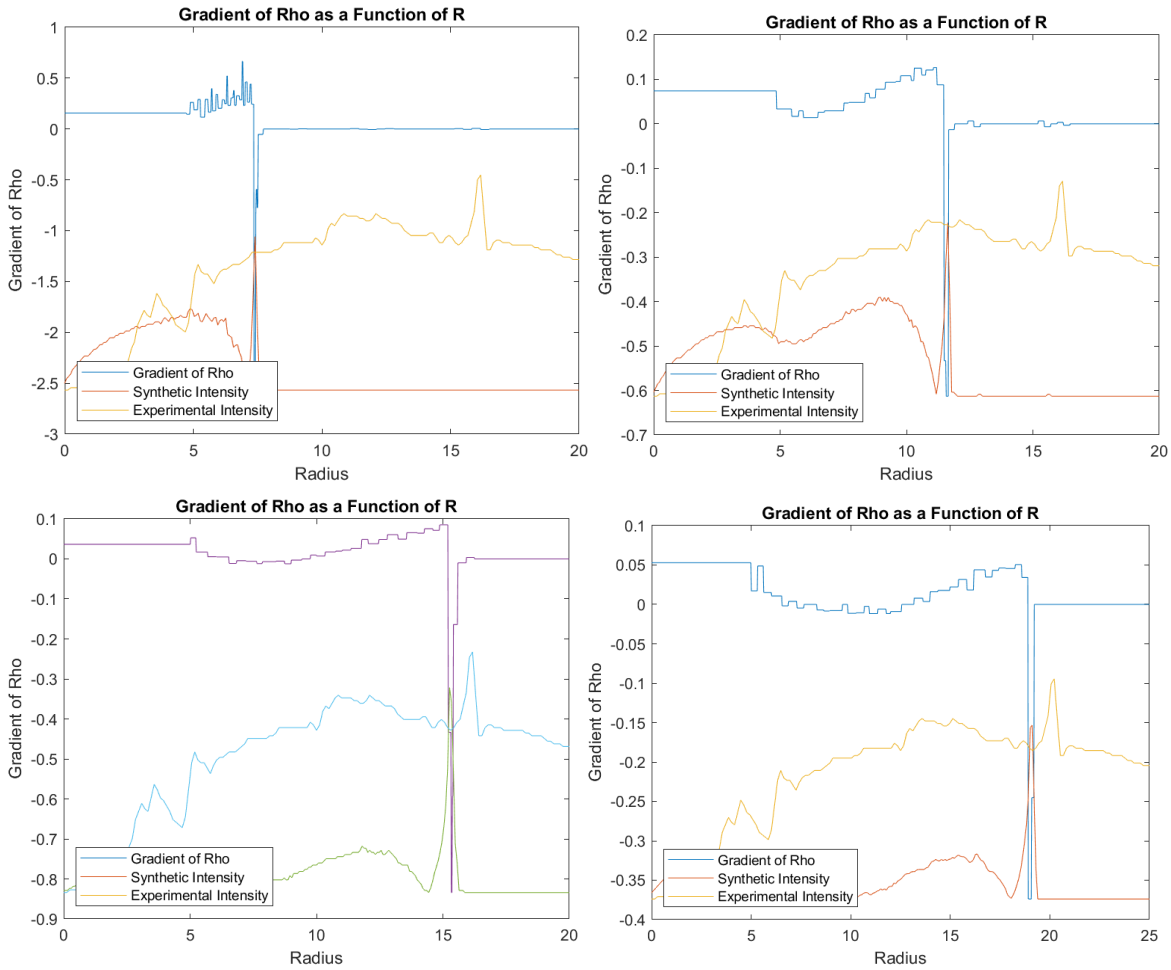
Spherical Blast Horizontal Knife Edge Plots (Times from 10 - 40 us)



Cylindrical Blast Adjust Circular Cutoff Plots (Increasing Times)



Spherical Blast Adjusted Inverted Circular Cutoff Plots (Times from 10 - 40 us)



A22. PowerPoints to Report Findings

See PowerPoints in the PowerPoints folder.

References

[1] H. Yan, R. University, R. Adelgren, U.S. Air Force Test Pilot School, M. Boguszko, G. Elliott, U. of I. at Urbana–Champaign, D. Knight, A. P. Khamseh, A. P. K. E. P. DeMauro, B. Sforzo, P.-Q. Elias, A. C. S. P. Schneider, N. Kianvashrad, L. Wermer, D. Heitmann, J. Sawyer, M. G.-A. D. Knight, L. Yang, S. Brieschenk, E. Schuelein, H. Y. and D. Gaitonde, B. Gardarin, N. Glumac, R. G. Adelgren, H. Yan, S. A. D. Knight, and S. Aradag, "Laser Energy Deposition in Quiescent Air," *AIAA Journal*, 17-May-2012. [Online]. Available: https://arc.aiaa.org/doi/10.2514/2.1888. [Accessed: 10-Sep-2020].

[2] A. Rohatgi, *WebPlotDigitizer - Copyright 2010-2020 Ankit Rohatgi*. [Online]. Available: https://apps.automeris.io/wpd/. [Accessed: 10-Sep-2020].

[3] *Physically-Based Interactive Flow Visualization Based on Schlieren and Interferometry Experimental Techniques - IEEE Journals & Magazine*. [Online]. Available: https://ieeexplore.ieee.org/document/5732716. [Accessed: 10-Sep-2020].

[4] N. Plooster, "Shock Waves From Line Sources," *Welcome to OpenSky 2.0*, 01-Jan-1970. [Online]. Available: https://opensky.ucar.edu/islandora/object/technotes:26. [Accessed: 10-Sep-2020].

[5] *Online Graph Maker · Plotly Chart Studio*. [Online]. Available: https://chart-studio.plotly.com/create/. [Accessed: 10-Sep-2020].

[6] A. Attard, J. H. S. Beresh, M. J. J. Bergthorson, J. B. A. Bonanos, E. W. M. Born, V. P. C. Brownlee, L. K. O. Bruno, H. K. R. Burns, B. C. JM. Chacin, K. C. S. Chang, O. B. J. Chaubell, H. C. P. Dimotakis, A. I. S. Discetti, V. F. F. Ducros, G. M. A. Ferrante, A. R. T. Fric, C. Gauss, J. B. A. Gerrard, J. Goodman, A. G. G. Gori, L. Z. L. Guo, K. Halbach, G. Haller, M. M. E. Hasselbrink, G. S. D. Jonassen, S. L. S. Kawai, A. S. M. Kline, A. R. H. Liepmann, M. R. K. Lin, G. C. R. MacCormack, K. Mahesh, S. W. C. Meinhart, E. O'Neill, F. P. H. Pagendarm, A. Papoulis, G. C. D. Peterson, R. Sampson, J. D. J. Santiago, F. B. J. Schetz, G. Settles, P. S. M. Shur, R. B. R. Srinivasan, G. C. P. Subbareddy, N. S. E. Touber, J. S. W. VanLerberghe, L. Vasiliev, S. Y. V. Vuorinen, Z. W. H. Wang, G. C. M. Wright, I. C. Z. Xie, and L. Yates, "Simulating schlieren and shadowgraph images from LES data," *Experiments in Fluids*, 01-Jan-1984. [Online]. Available: https://link.springer.com/article/10.1007/s00348-019-2774-6. [Accessed: 10-Sep-2020].